

Unifying Approach to Uniform Expressivity of Graph Neural Networks

Jonni Virtema

University of Glasgow, UK

September 8, 2026

Liverpool Discrete Mathematics Colloquium

Further details: **Ask** during the talk, after the talk, or **read the paper**

- Unifying approach to uniform expressivity of graph neural networks
23rd Int. Conference on Principles of Knowledge Representation and Reasoning, KR 2026.
(Joint work with Huan Luo.)
- ESSAI 26 course slides for more verbose introduction:
<https://www.virtema.fi/essai26.html>
Recurrent GNNs: The Power of Iteration (jointly with Floris Geerts).

Understanding limits of GNNs via connections to logic

- This is **descriptive complexity** of Graph Neural Networks.
- We focus on node classifiers of graphs.
 - **Input:** Labelled graph.
 - **Output:** A subset of nodes (i.e., label each node with 0 or 1).

Understanding limits of GNNs via connections to logic

- This is **descriptive complexity** of Graph Neural Networks.
- We focus on node classifiers of graphs.
 - **Input:** Labelled graph.
 - **Output:** A subset of nodes (i.e., label each node with 0 or 1).
- Examples with graphs or graph databases.
 - Compute **dominating set** of a graph.
 - In social network, decide whether a person is **likely to buy a product**.
 - Decide automatically whether to **grant a loan**, or **approve an ILR application**.

Understanding limits of GNNs via connections to logic

- This is **descriptive complexity** of Graph Neural Networks.
- We focus on node classifiers of graphs.
 - **Input:** Labelled graph.
 - **Output:** A subset of nodes (i.e., label each node with 0 or 1).
- Examples with graphs or graph databases.
 - Compute **dominating set** of a graph.
 - In social network, decide whether a person is **likely to buy a product**.
 - Decide automatically whether to **grant a loan**, or **approve an ILR application**.
- Formulae of **modal logic**, or **FO with one free variable** define node properties.
- To understand limitations of GNN architectures, we may want to find precise logical characterisations!

Understanding limits of GNNs via connections to logic

- This is **descriptive complexity** of Graph Neural Networks.
- We focus on node classifiers of graphs.
 - **Input:** Labelled graph.
 - **Output:** A subset of nodes (i.e., label each node with 0 or 1).
- Examples with graphs or graph databases.
 - Compute **dominating set** of a graph.
 - In social network, decide whether a person is **likely to buy a product**.
 - Decide automatically whether to **grant a loan**, or **approve an ILR application**.
- Formulae of **modal logic**, or **FO with one free variable** define node properties.
- To understand limitations of GNN architectures, we may want to find precise logical characterisations!
- There is a **GNN that computes** a **node property P** if and only if P is **definable in a logic**.

Understanding limits of GNNs via connections to logic

- This is **descriptive complexity** of Graph Neural Networks.
- We focus on node classifiers of graphs.
 - **Input:** Labelled graph.
 - **Output:** A subset of nodes (i.e., label each node with 0 or 1).
- Examples with graphs or graph databases.
 - Compute **dominating set** of a graph.
 - In social network, decide whether a person is **likely to buy a product**.
 - Decide automatically whether to **grant a loan**, or **approve an ILR application**.
- Formulae of **modal logic**, or **FO with one free variable** define node properties.
- To understand limitations of GNN architectures, we may want to find precise logical characterisations!
- There is a **GNN that computes** a **node property P** if and only if P is **definable in a logic**.
- **No guarantees that this GNN can be trained!**
- **No guarantees that this property cannot be approximated by a GNN practice!**

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
 - **Local model** of distributed computing corresponds to **graded modal logic** on degree bounded graphs. Send: Broadcast, Receive: Multiset. PODC 2012.
 - FO-definable $\cap$ **AC(R)-GNNs** are those definable in **GML** ($\approx$ + global modality). Barcelo et al. 2020.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
 - **Local model** of distributed computing corresponds to **graded modal logic** on degree bounded graphs. Send: Broadcast, Receive: Multiset. PODC 2012.
 - FO-definable $\cap$ **AC(R)-GNNs** are those definable in **GML** ($\approx$ + global modality). Barcelo et al. 2020.
 - K-hop subgraph GNNs related to a variant of WL (no logical part). Chen, Zhang, Wang 2025.
 - Bounded Counting GNNs and Fragments of FOL. Cuenca Grau, Feng, and Wałęga 2026.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
 - **Local model** of distributed computing corresponds to **graded modal logic** on degree bounded graphs. Send: Broadcast, Receive: Multiset. PODC 2012.
 - FO-definable $\cap$ **AC(R)-GNNs** are those definable in **GML** ($\approx$ + global modality). Barcelo et al. 2020.
 - K-hop subgraph GNNs related to a variant of WL (no logical part). Chen, Zhang, Wang 2025.
 - Bounded Counting GNNs and Fragments of FOL. Cuenca Grau, Feng, and Wałęga 2026.
 - Many others that vary GNNs and WLs, some even connect to a variant of pebble games.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
- Many of the results and proofs have common ingredients:
 1. Vary how and what data is communicated as an input to a GNN-layer.
 2. Define a variant of WL-algorithm (or bisimulation) for that communication.
 3. Show that the GNNs are invariant for that WL variant.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
- Many of the results and proofs have common ingredients:
 1. Vary how and what data is communicated as an input to a GNN-layer.
 2. Define a variant of WL-algorithm (or bisimulation) for that communication.
 3. Show that the GNNs are invariant for that WL variant.
 4. Define a (modal) logic with modalities that correspond to that style of communication.
 5. Show that every finite layer WL-equivalence class is definable in the logic.
 6. Show that the GNN architecture can recursively evaluate the logical formulas.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
- Many of the results and proofs have common ingredients:
 1. Vary how and what data is communicated as an input to a GNN-layer.
 2. Define a variant of WL-algorithm (or bisimulation) for that communication.
 3. Show that the GNNs are invariant for that WL variant.
 4. Define a (modal) logic with modalities that correspond to that style of communication.
 5. Show that every finite layer WL-equivalence class is definable in the logic.
 6. Show that the GNN architecture can recursively evaluate the logical formulas.
- Techniques used from model theory of modal logic + formula evaluation à la Barcelo et al.
 - Van Benthem-Rosen characterisation of ML as the bis. sim. inv. frag. of FO ('83 & '97).
 - Many variations of above exists with respect to various modal logics.

What was our starting point

- The past 5+ years has seen emergence of results relating logics to GNNs.
- Many of the results and proofs have common ingredients:
 1. Vary how and what data is communicated as an input to a GNN-layer.
 2. Define a variant of WL-algorithm (or bisimulation) for that communication.
 3. Show that the GNNs are invariant for that WL variant.
 4. Define a (modal) logic with modalities that correspond to that style of communication.
 5. Show that every finite layer WL-equivalence class is definable in the logic.
 6. Show that the GNN architecture can recursively evaluate the logical formulas.
- Techniques used from model theory of modal logic + formula evaluation à la Barcelo et al.
 - Van Benthem-Rosen characterisation of ML as the bis. sim. inv. frag. of FO ('83 & '97).
 - Many variations of above exists with respect to various modal logics.
- A caveat:
 - **Infinitary** characterisations follow then more-or-less directly.
 - The number of equivalence classes for 5. should be finite, so that every combination becomes definable. Common solution: Restrict to **bounded size** or **bounded degree** graphs.

What did we want to do?

- Can (many of) the **different characterisations** be done in a **unified manner**?
- Formulate a **meta theorem** yielding a **family of logical characterisations** of GNNs.

What did we want to do?

- Can (many of) the **different characterisations** be done in a **unified manner**?
- Formulate a **meta theorem** yielding a **family of logical characterisations** of GNNs.
- What would that entail?
 - A general framework that governs what GNN-layers aggregate.
 - A canonical way to generate needed WL-algorithms and modal logic.
 - A general characterisation proof strategy.

What did we want to do?

- Can (many of) the **different characterisations** be done in a **unified manner**?
- Formulate a **meta theorem** yielding a **family of logical characterisations** of GNNs.
- What would that entail?
 - A general framework that governs what GNN-layers aggregate.
 - A canonical way to generate needed WL-algorithms and modal logic.
 - A general characterisation proof strategy.
- Our design philosophy:
 - Logicians: **The logics and notions of bisimulation look natural and nice! Still, we could be more general and abstract!**
 - Non-logicians: **The GNN aggregation looks simple! No logic here!**

What did we want to do?

- Can (many of) the **different characterisations** be done in a **unified manner**?
- Formulate a **meta theorem** yielding a **family of logical characterisations** of GNNs.
- What would that entail?
 - A general framework that governs what GNN-layers aggregate.
 - A canonical way to generate needed WL-algorithms and modal logic.
 - A general characterisation proof strategy.
- Our design philosophy:
 - Logicians: **The logics and notions of bisimulation look natural and nice! Still, we could be more general and abstract!**
 - Non-logicians: **The GNN aggregation looks simple! No logic here!**
- **Did we reach the goal?** The work was presented in KR 2026. Hence, people who like introducing logics for KR applications were satisfied!

Overview



For standard message passing (Barceló et al. 2020):

AC-GNNs $\leftrightarrow$ 1-WL $\leftrightarrow$ Graded bisimulation $\leftrightarrow$ GML

Our approach

We abstract this correspondence using *templates*: small rooted patterns that govern where messages are aggregated from.

GNNs as Iterated Graph Transformations

Input: a labelled directed graph

$$G = (V, E, \lambda), \quad \lambda(v) \in \mathbb{R}^d.$$

A standard aggregate-combine GNN updates synchronously:

$$\lambda'(v) := \text{comb}'\left(\lambda'^{-1}(v), \text{agg}'\left(\{\lambda'^{-1}(u)\}_{u \in N(v)}\right)\right),$$

where

$$N(v) := \{u \in V \mid (v, u) \in E\} \quad (\text{out neighbours of } v).$$

- Each layer computes a graph transformation.
- After L layers, $\text{cls}(\lambda^L(v))$ gives a node classifier.
- We focus on Boolean node classifiers, i.e. unary graph queries.

Expressivity viewpoint

Ignore training; ask which node classifiers can be represented by the architecture.

Classical correspondence: AC-GNNs, 1-WL, and GML

1-WL / colour refinement

$$\text{col}^l(v) := \text{HASH}\left(\text{col}^{l-1}(v), \{\!\!\{\text{col}^{l-1}(u)\}\!\!\}_{u \in N(v)}\right).$$

- Same distinguishing power as AC-GNNs (Morris et al. 2019; Xu et al. 2019).
- Corresponds to graded bisimulation.

Graded modal logic

$$\Diamond^{\geq c} \varphi$$

with

$$(G, v) \models \Diamond^{\geq c} \varphi$$

iff v has at least c successors satisfying φ .

- Neighbour aggregation corresponds to graded modalities.

aggregate over neighbours

$\approx$

count successors satisfying formulas

Why Go Beyond Standard Message Passing?

Standard AC-GNNs only aggregate over immediate neighbours.

Limitations

- Many cycle properties are invisible.
- Expressivity is bounded by 1-WL.
- Rich local structure is not directly available.

Architectural responses

- subgraph GNNs,
- cycle-counting GNNs,
- homomorphism-pattern counts,
- non-neighbour aggregation.

Our abstraction

Messages do not have to come from edges. They may come from embeddings of rooted templates.

Templates

Definition

A *template* is a structure

$$T = (V, E^+, E^-, r),$$

where r is the root, E^+ are required edges, and E^- are required non-edges.



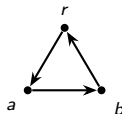
Edge template

$$E^+ = \{(r_1, a)\}, E^- = \emptyset$$



Non-edge template

$$E^+ = \emptyset, E^- = \{(r_2, a)\}$$



Triangle template

$$E^+ = \{(r, a), (a, b), (b, r)\}, E^- = \emptyset$$

Solid arrows are elements of E^+ ; dashed arrows are elements of E^- .

Template Embeddings

A template embedding of $T = (V, E^+, E^-, r)$ into a pointed graph (G, w) is an injective map

$$f : V \rightarrow V_G$$

such that

$$f(r) = w,$$

and for all $u, v \in V$:

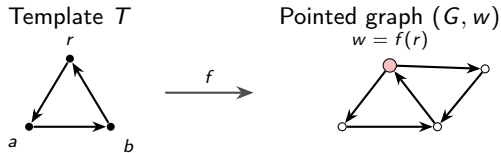
$$(u, v) \in E^+ \Rightarrow (f(u), f(v)) \in E_G,$$

$$(u, v) \in E^- \Rightarrow (f(u), f(v)) \notin E_G.$$

We write:

$$\text{emb}(T, (G, w))$$

for the set of all such embeddings.



Template GNNs

For a fixed template T , node v aggregates over all embeddings:

$$f \in \text{emb}(T, (G, v)).$$

Each embedding gives a labelled copy of the template:

$$(T, \lambda_f'^{-1}), \quad \lambda_f'^{-1}(u) := \lambda'^{-1}(f(u)).$$

Unary $\mathcal{T}$ -GNN update (one template per layer)

$$\lambda'(v) := \text{comb}' \left(\lambda'^{-1}(v), \underbrace{\text{agg}' \left\{ \underbrace{\text{agg}'_{T'}(T', \lambda_f'^{-1})}_{\text{template aggregation}} \mid f \in \text{emb}(T', (G, v)) \right\}}_{\text{outer aggregation}} \right).$$

- **Template aggregation** $\text{agg}_{T'}$: turns one labelled template occurrence into a message vector.
- **Outer aggregation** agg : collects the multiset of all such messages.
- **Combine** comb : updates the feature vector of the root node.

Template GNNs

For a fixed template T , node v aggregates over all embeddings:

$$f \in \text{emb}(T, (G, v)).$$

Each embedding gives a labelled copy of the template:

$$(T, \lambda_f'^{-1}), \quad \lambda_f'^{-1}(u) := \lambda'^{-1}(f(u)).$$

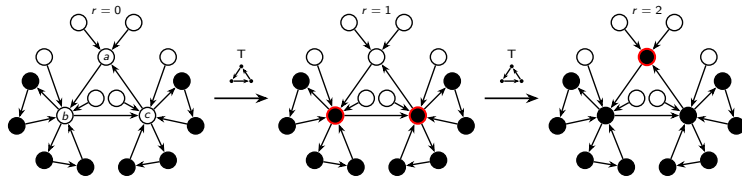
Unary $\mathcal{T}$ -GNN update (one template per layer)

$$\lambda'(v) := \text{comb}' \left(\lambda'^{-1}(v), \underbrace{\text{agg}' \left\{ \underbrace{\text{agg}'_{T'}(T', \lambda_f'^{-1})}_{\text{template aggregation}} \mid f \in \text{emb}(T', (G, v)) \right\}}_{\text{outer aggregation}} \right).$$

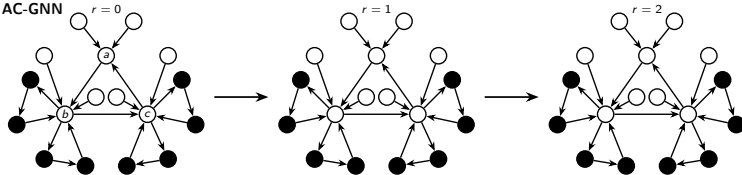
- **Template aggregation** $\text{agg}_{T'}$: turns one labelled template occurrence into a message vector.
- **Outer aggregation** agg : collects the multiset of all such messages.
- **Combine** comb : updates the feature vector of the root node.
- **Example** Edge (+ non-edge) template gives AC(R)-GNNs.

Example: Triangle-governed Message Passing

$\mathcal{T}$ -GNN



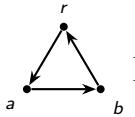
AC-GNN



$\mathcal{T}$ -GNN: b, c receive $\{\{\triangle_{\text{white}}, \triangle_{\text{black}}, \triangle_{\text{black}}\}\} \rightarrow \{\{\circ, \bullet, \bullet\}\} \rightarrow b, c : \bullet \rightarrow a : \bullet$

AC-GNN: b, c receive $\{\{\circ, \circ, \circ, \bullet, \bullet\}\} \rightsquigarrow$ no change

Existing Architectures as Template GNNs

Architecture	Template family	Consequence
AC-GNN AC ⁺ -GNN	$\{ r_1 \xrightarrow{\quad} a \}$ $\{ r_1 \xrightarrow{\quad} a, r_2 \dashrightarrow a \}$	standard message passing neighbours and non-neighbours
Triangle GNN	 $\{ \text{triangle}(a, b, r) \}$	triangle-context aggregation
k -hop subgraph GNN Pattern-enriched GNNs	rooted templates of radius k selected finite $\mathcal{T}$	subgraph-aware updates pattern-sensitive aggregation

Unifying view

Once an architecture is described by a finite template family $\mathcal{T}$, our framework supplies:

$\mathcal{T}$ -GNNs, $\mathcal{T}$ -WL, $\mathcal{T}$ -bisimulation, $\text{GML}(\mathcal{T})$.

$\mathcal{T}$ -WL Algorithm

The standard 1-WL update is:

$$\text{col}'(v) := \text{HASH}\left(\text{col}'^{-1}(v), \{\{\text{col}'^{-1}(u)\}_{u \in N(v)}\}\right).$$

$\mathcal{T}$ -WL update

For $\mathcal{T} = \{T_1, \dots, T_n\}$:

$$\begin{aligned} \text{col}'(v) := \text{HASH}\Big(&\text{col}'^{-1}(v), \{\{(T_1, \text{col}'^{-1}_f) \mid f \in \text{emb}(T_1, (G, v))\}\}, \\ &\dots, \{\{(T_n, \text{col}'^{-1}_f) \mid f \in \text{emb}(T_n, (G, v))\}\}\Big). \end{aligned}$$

- With the edge template, this is exactly 1-WL.
- With all rooted radius- k templates, it corresponds to k -hop subgraph WL.

Graded $\mathcal{T}$ -Bisimulation

Two nodes (G, v) and (G', v') are graded l - $\mathcal{T}$ -bisimilar if:

1. They agree on all proposition symbols.
2. They agree at depth $l - 1$:

$$(v, v') \in Z_{l-1}.$$

3. For every template $T \in \mathcal{T}$, each embedding f of T at v be matched with an embedding f' of T at v' such that all corresponding positions are $(l - 1)$ - $\mathcal{T}$ -bisimilar:

$$(f(u), f'(u)) \in Z_{l-1} \quad \text{for every } u \in T,$$

and vice versa.

For counting bound $c \geq 1$, the l - c - $\mathcal{T}$ -bisimulation requires matching up to c embeddings.

Template WL and bisimulation coincide

$$(G', v') \sim_{\mathcal{T}}^l (G, v) \Leftrightarrow \text{col}^l(v) = \text{col}^l(v').$$

Analogously for the c -bounded version.

Graded Template Modal Logic $\text{GML}(\mathcal{T})$

Let $\mathcal{T}$ be a finite set of templates.

Syntax

$$\varphi := p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \langle T \rangle^{\geq j}(\varphi_1, \varphi_2, \dots, \varphi_{n_T}),$$

where $T \in \mathcal{T}$, $n_T = |V| - 1$, and $1, \dots, n_T$ index the non-root vertices (0 is the root).

Semantics

$$(G, v) \models \langle T \rangle^{\geq j}(\varphi_1, \varphi_2, \dots, \varphi_{n_T})$$

iff

$$|\{f \in \text{emb}(T, (G, v)) \mid (G, f(i)) \models \varphi_i, \text{ for } 1 \leq i \leq n_T\}| \geq j.$$

- Edge template gives ordinary graded modality:

$$\langle T_{\text{edge}} \rangle^{\geq j}(\varphi) \equiv \Diamond^{\geq j}\varphi.$$

- Triangle template counts triangle embeddings satisfying position-wise formulae.

Graded Template Modal Logic $\text{GML}(\mathcal{T})$

Let $\mathcal{T}$ be a finite set of templates.

Syntax

$$\varphi := p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \langle T \rangle^{\geq j}(\varphi_1, \varphi_2, \dots, \varphi_{n_T}),$$

where $T \in \mathcal{T}$, $n_T = |V| - 1$, and $1, \dots, n_T$ index the non-root vertices (0 is the root).

Semantics

$$(G, v) \models \langle T \rangle^{\geq j}(\varphi_1, \varphi_2, \dots, \varphi_{n_T})$$

iff

$$|\{f \in \text{emb}(T, (G, v)) \mid (G, f(i)) \models \varphi_i, \text{ for } 1 \leq i \leq n_T\}| \geq j.$$

- Edge template gives ordinary graded modality:

$$\langle T_{\text{edge}} \rangle^{\geq j}(\varphi) \equiv \Diamond^{\geq j}\varphi.$$

- Triangle template counts triangle embeddings satisfying position-wise formulae.
- Template modality is essentially one that is definable by a QF-formula.

Main Result

Metatheorem

Let $\mathcal{T}$ be a finite set of templates. Boolean node classifiers definable by bounded $\mathcal{T}$ -GNNs are exactly the node classifiers definable in $\text{GML}(\mathcal{T})$.

In bounded GNNs the outer aggregation function is a bounded function!

More specifically:

1. **GNN to logic.** If $\mathcal{N}$ is a c -bounded $\mathcal{T}$ -GNN with l layers, then there is

$$\varphi_{\mathcal{N}} \in \text{GML}(\mathcal{T})$$

of modal depth l and counting bound c that captures $\mathcal{N}$.

2. **Logic to GNN.** If $\varphi \in \text{GML}(\mathcal{T})$ with syntactic depth l and counting bound c , then there is a c -bounded $\mathcal{T}$ -GNN $\mathcal{N}_{\varphi}$ with l layers that captures φ .

$\text{bounded } \mathcal{T}\text{-GNNs} \equiv \text{GML}(\mathcal{T})$

Why Bounded Counting?

For $c \geq 1$, a multiset function f is c -bounded if it cannot distinguish multiplicities above c :

$$f(A) = f(A \upharpoonright c),$$

where $A \upharpoonright c$ caps all multiplicities larger than c at c .

In GNNs

The outer aggregation function only sees counts up to c .

In logic

A formula has finite counting threshold:

$$\langle T_{edge} \rangle^{\geq j}, \quad j \leq c.$$

Key property: For fixed $I, c, \mathcal{T}$, the relation $\sim_{\mathcal{T}}^{I, c}$ has finite index, enabling characteristic formulae.

Proof Idea I: GNNs are Bisimulation Invariant

Proposition

Let $\mathcal{N}$ be a c -bounded L -layer $\mathcal{T}$ -GNN. If

$$(G', v') \sim_{\mathcal{T}}^{l,c} (G, v),$$

then after l layers:

$$\lambda^l(v) = \lambda^l(v').$$

Induction on layers.

- Base case: labels agree.
- Inductive step:
 - template bisimulation matches embeddings;
 - corresponding template positions have equal feature vectors;
 - c -bounded aggregation sees the same multiset up to c ;
 - hence `comb` produces the same next feature vector.

Then accepted classes are l - c - $\mathcal{T}$ invariant, definable by disjunctions of characteristic formulae.

Proof Idea II: GNNs Simulate Formulae

Given $\varphi \in \text{GML}(\mathcal{T})$, construct a $\mathcal{T}$ -GNN with one coordinate per subformula:

$$\lambda^i(v)_i = 1 \quad \Leftrightarrow \quad (G, v) \models \varphi_i.$$

Boolean cases

Implemented by the combination function:

$$\neg, \quad \wedge$$

using thresholded linear operations.

Modal case

For

$$\varphi_i = \langle T_{edge} \rangle^{\geq j} (\psi_1, \dots, \psi_n),$$

template aggregation checks whether an embedding satisfies all ψ_k at the corresponding positions, and outer aggregation counts satisfying embeddings up to threshold j .

Limitations and Future Work

- Is the restriction to bounded GNNs necessary?
 - **Yes:** Restricting to FO-definable properties does not work. Reason: Already in ACR-GNNs unbounded counting increases expressivity past GML (Hauke and Wałęga, 2026).
 - **Probably not:** Add counting features to template GML. E.g., $\langle T \rangle^{=x}(\psi_1, \dots, \psi_n)$ as a counting term.

Limitations and Future Work

- Is the restriction to bounded GNNs necessary?
 - **Yes:** Restricting to FO-definable properties does not work. Reason: Already in ACR-GNNs unbounded counting increases expressivity past GML (Hauke and Wałęga, 2026).
 - **Probably not:** Add counting features to template GML. E.g., $\langle T \rangle^{=x}(\psi_1, \dots, \psi_n)$ as a counting term.
- Does it make sense to consider recursive GNNs?
 - **Probably:** What variant of recursion would be the most suitable?

Limitations and Future Work

- Is the restriction to bounded GNNs necessary?
 - **Yes:** Restricting to FO-definable properties does not work. Reason: Already in ACR-GNNs unbounded counting increases expressivity past GML (Hauke and Wałęga, 2026).
 - **Probably not:** Add counting features to template GML. E.g., $\langle T \rangle^{=x}(\psi_1, \dots, \psi_n)$ as a counting term.
- Does it make sense to consider recursive GNNs?
 - **Probably:** What variant of recursion would be the most suitable?
- Does it make sense to generalise further?
 - **Probably:** But with care!
- Further direction: Shift from Boolean classification to **real valued functions**. Shift **from Graphs to relational structures** (Carsten will talk about this more).

Relationship with Deep Homomorphism Networks

- Closely related model by Maehara and NT (NeurIPS 2024).
- Their GNN model has similar ingredients to ours.
 - A rooted pattern graph (T, μ) with a transformation $\mu_v: \mathbb{R}^d \rightarrow \mathbb{R}^d$, for each node $v \in T$.
 - A set $\text{Hom}(T, G)$ of homomorphisms from T to G (similar to template embeddings).
- Generalised homomorphism number with features replaces template aggregation:

$$\text{hom}((T, \mu), (G, \lambda)) := \sum_{\pi \in \text{Hom}(T, G)} \prod_{v \in T} \mu_v(\lambda_{\pi(v)})$$

Relationship with Deep Homomorphism Networks

- Closely related model by Maehara and NT (NeurIPS 2024).
- Their GNN model has similar ingredients to ours.
 - A rooted pattern graph (T, μ) with a transformation $\mu_v: \mathbb{R}^d \rightarrow \mathbb{R}^d$, for each node $v \in T$.
 - A set $\text{Hom}(T, G)$ of homomorphisms from T to G (similar to template embeddings).
- Generalised homomorphism number with features replaces template aggregation:

$$\text{hom}((T, \mu), (G, \lambda)) := \sum_{\pi \in \text{Hom}(T, G)} \prod_{v \in T} \mu_v(\lambda_{\pi(v)})$$

- Template embeddings are replaced with **homomorphisms**.
- Outer aggregation here is always the **sum**.
- Inner template aggregation is replaced with a **product of π_v transformed features**.

Relationship with Deep Homomorphism Networks

- Closely related model by Maehara and NT (NeurIPS 2024).
- Their GNN model has similar ingredients to ours.
 - A rooted pattern graph (T, μ) with a transformation $\mu_v: \mathbb{R}^d \rightarrow \mathbb{R}^d$, for each node $v \in T$.
 - A set $\text{Hom}(T, G)$ of homomorphisms from T to G (similar to template embeddings).
- Generalised homomorphism number with features replaces template aggregation:

$$\text{hom}((T, \mu), (G, \lambda)) := \sum_{\pi \in \text{Hom}(T, G)} \prod_{v \in T} \mu_v(\lambda_{\pi(v)})$$

- Template embeddings are replaced with **homomorphisms**.
- Outer aggregation here is always the **sum**.
- Inner template aggregation is replaced with a **product of π_v transformed features**.
- Our model is more general, but most likely the models coincide with respect to node classifiers of finite graphs.

Relationship with Deep Homomorphism Networks

- Closely related model by Maehara and NT (NeurIPS 2024).
- Their GNN model has similar ingredients to ours.
 - A rooted pattern graph (T, μ) with a transformation $\mu_v: \mathbb{R}^d \rightarrow \mathbb{R}^d$, for each node $v \in T$.
 - A set $\text{Hom}(T, G)$ of homomorphisms from T to G (similar to template embeddings).
- Generalised homomorphism number with features replaces template aggregation:

$$\text{hom}((T, \mu), (G, \lambda)) := \sum_{\pi \in \text{Hom}(T, G)} \prod_{v \in T} \mu_v(\lambda_{\pi(v)})$$

- Template embeddings are replaced with **homomorphisms**.
- Outer aggregation here is always the **sum**.
- Inner template aggregation is replaced with a **product of π_v transformed features**.
- Our model is more general, but most likely the models coincide with respect to node classifiers of finite graphs.
- Carsten will talk about Deep Homomorphism Networks in more detail.

Takeaways

1. Templates abstract the structural patterns over which GNNs aggregate.
2. For every finite template family $\mathcal{T}$, we obtain matching notions:

$$\mathcal{T}\text{-GNN}, \quad \mathcal{T}\text{-WL}, \quad \mathcal{T}\text{-bisimulation}, \quad \text{GML}(\mathcal{T}).$$

3. A recurring expressivity proof recipe becomes a reusable metatheorem:

$\text{bounded } \mathcal{T}\text{-GNNs} \equiv \text{GML}(\mathcal{T})$

Takeaways

1. Templates abstract the structural patterns over which GNNs aggregate.
2. For every finite template family $\mathcal{T}$, we obtain matching notions:

$$\mathcal{T}\text{-GNN}, \quad \mathcal{T}\text{-WL}, \quad \mathcal{T}\text{-bisimulation}, \quad \text{GML}(\mathcal{T}).$$

3. A recurring expressivity proof recipe becomes a reusable metatheorem:

$\text{bounded } \mathcal{T}\text{-GNNs} \equiv \text{GML}(\mathcal{T})$

Thank you.